Segment routing under planned link outages: a MILP study of the Orange T-ASR problem

[Full name 1 – Student ID] [Full name 2 – Student ID]

Mathematical Modelling project, SoICT

October 2026

Abstract

We study the T-Adaptive Segment Routing problem proposed by Orange for the EURO/ROADEF 2026 challenge. Each traffic demand of an IP network has to be assigned a segment path, i.e. a short list of waypoints between which packets follow shortest paths, so that the most loaded link stays as low as possible, both in the nominal state and when a link is switched off for maintenance. Following the course guidelines we build a mixed-integer linear model for the nominal period, then extend it to two periods with a budget on the number of routing changes. Split ratios are computed from shortest-path counts. The model is solved with JuMP and HiGHS on the 20 instances of set A under a 15 GiB memory limit. The full model can only be built for the five smallest instances, so for the others we solve a restricted model with a small set of candidate waypoints per demand, and we use a multi-commodity flow relaxation as an independent lower bound. Segment routing lowers the maximum link utilization by about 21 % on average and by up to 80 %. Sixteen of the 40 runs are proven optimal for the original problem.

1 Introduction

Orange operates large IP/MPLS backbones with thousands of routers and tens of thousands of traffic demands. In such networks packets are usually forwarded along shortest paths computed by the interior gateway protocol from administrative link weights. When several shortest paths exist, the traffic is split between them (Equal-Cost Multi-Path, ECMP). Routing on shortest paths is simple to operate, but the only way to steer traffic is to change the weights, and finding weights that avoid congestion is a hard problem in its own right [3]. A weight change also moves many demands at once, which is not what an operator wants during a maintenance window.

Segment Routing (SR) [2] gives a finer control. The ingress router writes into the packet header a list of intermediate nodes, called waypoints or node segments. The packet travels on shortest paths from the source to the first waypoint, then to the next one, and so on until the destination. With one or two waypoints a demand can be pushed away from a congested area without touching the link weights [4]. The price is that every waypoint is a configuration item that someone has to deploy and maintain.

The challenge problem, called T-Adaptive Segment Routing (T-ASR) [1], adds a time dimension. Routing is planned over a horizon of periods. Period 0 is the nominal network; in later periods some links are taken down for planned interventions, and the traffic matrix may change as well. For each period one has to choose a segment path for every demand, with at most *maxSeg* segments, while the number of segment changes between consecutive periods is bounded by a budget. The official objective minimizes all link loads in lexicographic order.

This report follows the steps given for the course project. We first compute the split ratios from the network data, then write and solve a model for period 0 alone (no budget), and finally extend it to the two periods of the set A instances with the budget constraint. Section 2 describes the data,

Table 1: Model parameters and their source files.

Symbol	Meaning	Source
$c(a)$, $\omega(a)$	capacity and IGP metric of arc a	`net.json`, fields `capacity`, `metric`
$r(i, j, a, t)$	share of the $i \to j$ flow carried by a in G_t	computed from the metrics (Sec. 3)
$\nu(d, t)$	volume of demand d in period t	`tm.json`, field `v[t]`
maxSeg	maximum number of segments	`scenario.json`, `max_segments`
$q(t)$, $\kappa(t)$	arcs down and change budget in period t	`scenario.json`, `interventions`, `budget`

Section 3 the split ratios and Section 4 the model. Section 5 explains how we solved it on large instances with limited memory, and Section 6 reports the experiments. We close with remarks on scalability and on what we would do next.

2 Problem description and data

The network is a directed graph $G = (V, A)$ with $n = |V|$ routers. Each arc $a \in A$ has a weight $\omega(a) > 0$, used to compute shortest paths, and a capacity $c(a) > 0$. A demand $d \in D$ is a pair (s_d, t_d) with a traffic volume $\nu(d, t) \geq 0$ in each period $t \in T = \{0, \ldots, h-1\}$. An intervention scenario gives, for each $t \geq 1$, the set $q(t) \subseteq A$ of arcs that are down, so the network of period t is $G_t = (V, A \setminus q(t))$. A segment path of d is a sequence $\langle s_d, w_1, \ldots, w_\ell, t_d \rangle$; its segments are the consecutive pairs $(s_d, w_1), \ldots, (w_\ell, t_d)$, and their number $\ell + 1$ may not exceed *maxSeg*. The distance between two routing schemes is the number of segments that appear in exactly one of them, and it may not exceed the budget $\kappa(t)$ between periods $t-1$ and t.

Table 1 lists where each parameter comes from. The set A benchmark has 20 instances with 20 to 400 nodes, 80 to 2000 arcs and 20 to 6000 demands (Table 2). All of them have two periods, *maxSeg* $= 6$, exactly one link down in period 1, and a budget between 1 and 90. A few details of the files mattered for the implementation. Nine instances carry the flag `multigraph: true`, but none of them actually contains parallel arcs, so a simple directed graph is enough. In setA-08 and setA-14 the metrics are not integers, which affects the counting of shortest paths (Section 3). Finally, the volumes of period 1 are not a scaled copy of period 0; some demands grow by a factor of more than 60, so period 1 differs from period 0 by more than the missing link.

3 Split ratios

The split ratio $r(i, j, a, t)$ is the fraction of the traffic sent from i to j that crosses arc a in G_t. It is defined for every ordered pair (i, j), since any pair may become a segment. Following the course notes we take

$$r(i, j, a, t) = \frac{\sigma_{ij}^a(t)}{\sigma_{ij}(t)}, \tag{1}$$

where σ_{ij} is the number of shortest $i \to j$ paths and σ_{ij}^a the number of those that use a. Both are obtained without enumerating paths. A Dijkstra run from i keeps, next to the distance label $d(v)$, a counter $\sigma(v)$: when an arc (u, v) gives a strictly shorter path, $\sigma(v)$ is reset to $\sigma(u)$, and when it gives a path of equal length, $\sigma(u)$ is added to $\sigma(v)$. Running it from every node yields the matrices d_{ij} and σ_{ij}; in Julia this is `dijkstra_shortest_paths` with `allpaths=true`. An arc $a = (u, v)$ lies on a shortest $i \to j$ path if and only if $d_{iu} + \omega(a) + d_{vj} = d_{ij}$. Such a path is a shortest $i \to u$ path, followed by a, followed by a shortest $v \to j$ path, and every combination of the two parts is a shortest path, so $\sigma_{ij}^a = \sigma_{iu}\sigma_{vj}$. Hence

$$r(i, j, a, t) = \begin{cases} \sigma_{iu}\,\sigma_{vj}/\sigma_{ij} & \text{if } d_{iu} + \omega(a) + d_{vj} = d_{ij}, \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

The arcs with $r > 0$ form the forwarding graph $FG(i, j)$. For every pair, $r(i, j, \cdot, t)$ must be a unit flow from i to j, and we check this flow conservation on every pair of every instance. The check caught a numerical problem. With non-integer metrics, two paths of the same length can differ in the last bit of their floating point sum, and the exact comparison used by Graphs.jl then misses some of them. On setA-08 and setA-14 the conservation error reached 1, meaning that part of the flow simply disappeared. In that case we recompute the counts with our own Dijkstra in which two lengths are equal when they agree up to a relative tolerance of 10^{-9}. After this the error is zero on all instances. The ratios are stored sparsely, pair by pair; for setA-20 there are 716 276 non-zero values in period 0 and the whole computation takes less than 4 seconds.

This definition differs from the one in the challenge description, where ECMP splits the flow evenly at every node of the forwarding graph. On the toy instance of the subject, for the demand (v_0, v_5) there are three shortest paths, one through v_3 only and two through v_4. Path counting gives $r = 1/3$ to arc (v_1, v_3) and $2/3$ to (v_1, v_4), while per-node ECMP gives $1/2$ to each. For the demand (v_2, v_5) both rules agree. We use path counting as required by the course; with it the toy instance has a no-waypoint MLU of 0.4167 instead of the 0.4375 quoted in the subject. To make sure the rest of the code is right, our test suite also implements the per-node rule and checks that the load computation then reproduces the figures of the subject exactly.

4 Mathematical model

Let $P_t = \{(i, j) \in V^2 : i \neq j,\ \sigma_{ij}(t) > 0\}$ be the pairs that can be used as segments in period t. The decision variables are

- $x_{ij}^{dt} \in \{0, 1\}$, equal to 1 if the segment path of demand d in period t contains the segment (i, j), for $(i, j) \in P_t$;
- $U \geq 0$, the maximum link utilization over all arcs and periods.

The model reads

$$\min \quad U$$

$$\text{s.t.} \quad \sum_{j:(i,j)\in P_t} x_{ij}^{dt} - \sum_{j:(j,i)\in P_t} x_{ji}^{dt} = \begin{cases} 1 & i = s_d \\ -1 & i = t_d \\ 0 & \text{otherwise} \end{cases} \qquad \forall d \in D,\ i \in V,\ t \in T, \tag{3}$$

$$\sum_{(i,j)\in P_t} x_{ij}^{dt} \leq maxSeg \qquad \forall d \in D,\ t \in T, \tag{4}$$

$$\sum_{d\in D} \sum_{(i,j)\in P_t} r(i, j, a, t)\, \nu(d, t)\, x_{ij}^{dt} \leq c(a)\, U \qquad \forall t \in T,\ a \in A \setminus q(t), \tag{5}$$

$$\sum_{d\in D} \sum_{(i,j)} |x_{ij}^{dt} - x_{ij}^{d,t-1}| \leq \kappa(t) \qquad \forall t \in T \setminus \{0\}, \tag{6}$$

$$x_{ij}^{dt} \in \{0, 1\}, \quad U \geq 0.$$

For step 6 of the project $T = \{0\}$ and constraint (6) is absent; for step 9 $T = \{0, 1\}$.

Why each constraint. Think of every pair $(i, j) \in P_t$ as a virtual arc of a complete graph. Constraint (3) asks the virtual arcs chosen for d to form a path from s_d to t_d; the inner nodes of that path are the waypoints. The number of virtual arcs on the path is the number of segments, which is what (4) limits. In (5) the traffic of d on a segment (i, j) is spread over the real arcs of $FG(i, j)$ according to r, and the sum over demands and segments is the flow on arc a. Dividing by $c(a)$ gives the load $\lambda(a, t)$, so (5) says $U \geq \lambda(a, t)$ for every arc that is up; at the optimum U equals the maximum load. Arcs in $q(t)$ get no constraint, and they never carry flow anyway because they are absent from G_t. Constraint (6) is the distance between the two routing schemes as defined in the subject.

Objective. The subject asks for the lexicographic minimum of all loads. Its first component is the maximum load, so minimizing U is the first level of that objective. Computing the full lexicographic optimum would require a sequence of up to $|A|\,|T|$ MILPs, each as large as ours, which is out of reach here. MLU is also the criterion most used in traffic engineering. With two periods the same variable U bounds the loads of both, so the objective is $\max(\mathrm{MLU}_0, \mathrm{MLU}_1)$.

Restricting the pairs. We only create x_{ij}^{dt} for $(i,j) \in P_t$. If j cannot be reached from i, all ratios of (i,j) are zero, and a variable on that pair would carry traffic for free. This happens in period 1 when the down arc disconnects some pairs, so the variables of such a pair are fixed to 0 in that period.

Cycles. Constraint (3) allows, besides the s_d–t_d path, cycles of virtual arcs that are disconnected from it. They are harmless. Since $r \geq 0$ and $\nu \geq 0$, removing a cycle can only decrease loads and the number of segments, so an optimal solution without cycles always exists and no subtour constraint is needed. When reading a solution we follow the chosen segments from s_d and drop anything not on the path.

Linearization of the budget. The usual way is to add $z \geq x^{dt} - x^{d,t-1}$ and $z \geq x^{d,t-1} - x^{dt}$ for each term and bound $\sum z$. We use the identity $|\Delta| = 2\Delta^+ - \Delta$ instead, which needs one row per term:

$$z_{ij}^d \geq x_{ij}^{d1} - x_{ij}^{d0}, \quad z_{ij}^d \geq 0, \quad b_d = \sum_{(i,j)} (2z_{ij}^d - x_{ij}^{d1} + x_{ij}^{d0}), \quad \sum_{d \in D} b_d \leq \kappa(1). \tag{7}$$

Since z is only bounded from below, a feasible z exists exactly when $\sum |\Delta| \leq \kappa$, so the formulation is exact. The per-demand counters b_d look redundant but they matter in practice. Written as a single row, the budget constraint contains every z variable of the model (hundreds of thousands of coefficients), and the HiGHS presolve got stuck on it: on setA-20 it ran for more than 20 minutes without honouring a 60-second time limit. With the b_d split, presolve takes about 5 seconds, and the step 9 model of setA-13, which had also blocked in presolve, is solved to optimality in 38 seconds.

Aggregated load rows. Written as in (5), the load rows have $|D| \cdot \sum_{(i,j)} |FG(i,j)|$ non-zeros, for example 23 million for setA-08. We introduce one continuous variable per segment and period, $y_{ij}^t = \sum_d \nu(d,t)\, x_{ij}^{dt}$, and write the load rows as $\sum_{(i,j)} r(i,j,a,t)\, y_{ij}^t \leq c(a)U$. This is only a renaming, but the count drops to $|D|\,|P_t| + \sum |FG(i,j)|$, i.e. 4.6 million for setA-08. Demands with zero volume in every period are removed from the model and routed directly.

Size. The model has $|D| \cdot |P_t| \leq |D|\, n(n-1)$ binary variables per period. The last column of Table 2 gives this number for step 9: from 30 400 for setA-01 to 1.9 billion for setA-20. Size, not difficulty, is the first obstacle on this benchmark.

5 Solution approach

All experiments had to fit in 15 GiB of memory. Measured on our runs, the full model takes roughly 2.5 KB per variable once HiGHS has loaded it, so only instances with at most a few million variables can be built. We used three ingredients.

Full model when it fits. When the full model has at most 2.5 million x variables (setA-01 to setA-05, except setA-04 in step 9, see below), we solve it directly. Two changes to the default settings were needed. First, the LP relaxation of these models is large and highly degenerate. On

setA-03 step 9 the dual simplex performed 752 481 iterations in one hour without finishing the root LP, whereas the interior point method solved it in 36 seconds; we therefore set HiGHS to use the interior point method for the LP at the root. Second, the primal heuristics of HiGHS rarely found anything better than the starting solution on these models. We start from the solution of the restricted model described next, solved with at most 10 % of the time limit. It is feasible for the full model and usually already very good.

Restricted model otherwise. For larger instances, demand d may only use waypoints from a candidate set W_d of K nodes, those with the smallest detour $d_{s_d w} + d_{w t_d} - d_{s_d t_d}$. The segments of d are the pairs of $\{s_d\} \cup W_d \cup \{t_d\}$, without arcs entering s_d or leaving t_d. We choose $K = \lfloor \sqrt{600\,000/(2|D|)} \rfloor - 1$, clipped to $[3, 30]$, which gives about 600 000 binary variables in step 9; the same K is used in step 6, so that the step 6 model is exactly the period-0 part of the step 9 model. Every solution of the restricted model is a valid routing, so its objective is an upper bound for the original problem. Its "optimal" status, however, only refers to the restricted space, and its dual bound says nothing about the original problem.

An independent lower bound. To judge the restricted solutions we solve a multi-commodity flow relaxation: each demand may split its traffic arbitrarily over all paths of G_t, and we minimize the maximum load. Any segment routing is such a flow, so the optimum of this LP (aggregated by source node, solved by the interior point method) is a valid lower bound on the T-ASR optimum. When a solution reaches this bound it is optimal for the original problem. The bound can be weak, because SR cannot split a demand freely.

Correctness checks. Every solution is decoded into waypoint lists and checked by code that does not use the model: each path is simple, has at most *maxSeg* segments, uses only pairs reachable in G_t, the budget is respected, and the MLU recomputed from the waypoints matches the objective value. The whole pipeline was first tested on the toy instance of the subject, where all values can be checked by hand.

6 Numerical experiments

6.1 Setting

The code is written in Julia 1.13.2 with JuMP 1.32.1 [7], HiGHS 1.15.1 (through HiGHS.jl 1.26.0) [6], Graphs.jl 1.15.0 and JSON3. Runs were made on a Linux server with an AMD EPYC 7742 processor. HiGHS used a single thread, random seed 1 and a relative gap tolerance of 10^{-4}. The time limit was 1800 s for step 6 and 3600 s for step 9. Each solve ran in its own process, and all processes together were confined to a Linux control group with a hard limit of 15 GiB, several solves sharing it at the same time. CPU time is measured around the call to `optimize!` only, after a warm-up run that removes Julia compilation time; it excludes the time to compute the ratios (at most 4 s) and to build the model (at most 95 s).

The project guidelines ask for averages over ten runs. Because of the available time we made a single run per instance and step, so the tables report one value and no standard deviation. Most instances are solved at the root node, where the seed has little influence, but the runs that hit the time limit could vary from one seed to another.

Table 2 describes the instances. Two observations help reading the results. The no-waypoint MLU ("baseline") is 1.0000 or very close to it for most instances and periods, which suggests that the volumes were scaled so that shortest-path routing just saturates the network; any value below 1 means that SR has removed the congestion. The multi-commodity flow bound is sometimes very

Table 2: Set A instances. κ: budget of period 1; down: id of the link removed in period 1; baseline: MLU without waypoints; MCF: multi-commodity flow lower bound (step 6: period 0; step 9: both periods); $|x|$: binary variables of the full step 9 model, in millions.

						Baseline		MCF bound		
Inst	$\|V\|$	$\|A\|$	$\|D\|$	κ	down	$t=0$	$t=1$	step 6	step 9	$\|x\|$ (M)
01	20	80	40	51	22	1.0000	0.5663	0.5604	0.5604	0.03
02	30	150	45	63	59	1.0000	1.0981	0.5388	0.6243	0.08
03	50	250	20	53	52	1.0000	0.6019	0.6337	0.6337	0.10
04	50	250	200	44	106	0.6918	1.0000	0.3817	0.4832	0.98
05	100	396	100	1	337	0.3333	1.0000	0.1155	0.2050	1.98
06	100	500	500	13	53	1.0000	0.7746	0.0986	0.0986	9.90
07	100	500	800	90	220	0.9080	1.0000	0.9080	0.9080	15.84
08	150	654	200	13	584	1.0000	0.7015	0.3189	0.3189	8.94
09	150	750	200	18	152	0.6111	1.0000	0.4254	0.8497	8.94
10	150	966	1000	1	962	1.0000	0.8913	0.0683	0.0683	44.70
11	200	1000	400	89	399	1.0000	0.7858	0.6131	0.7858	31.84
12	200	898	400	13	676	1.0001	0.8799	0.5789	0.8799	31.84
13	200	1000	2000	12	153	1.0000	0.9359	0.0389	0.0408	159.20
14	250	1108	600	13	901	1.0000	0.7356	0.4239	0.5074	74.70
15	250	1250	600	54	541	1.0000	0.8987	0.8252	0.8987	74.70
16	250	1452	4800	13	270	0.9672	1.0000	0.0386	0.0441	597.60
17	300	1270	2000	1	1074	1.0001	0.4445	0.4242	0.4242	358.80
18	300	1500	2000	89	735	0.6334	1.0000	0.6334	1.0000	358.80
19	300	1998	6000	13	581	1.0000	0.8852	0.0468	0.0468	1076.40
20	400	2000	6000	90	1631	1.0000	0.8408	0.6482	0.6482	1915.20

low (0.04 for setA-13, 16 and 19), which already tells us that it will not certify much on those instances.

6.2 Period 0 only (step 6)

Table 3 gives the results for period 0. Status, gap and node count are those reported by HiGHS for the model that was solved, with gap $= 1 - \mathrm{LB}/\mathrm{UB}$. A star marks the solutions that are provably optimal for the original problem, either because the full model was solved to optimality or because the objective equals the MCF bound.

The full model is solved to optimality on the three smallest instances, in less than 7 minutes. On setA-04 and setA-05 (475 000 and 990 000 binaries) HiGHS spends the whole time limit on the root node. For setA-05 the starting solution from the restricted model already brings the MLU from 0.333 to 0.132, and the root bound shows that it is within 12.8 % of the optimum. Note that HiGHS overran the limit on this instance by about 20 minutes; the interior point method only checks the clock between iterations, and one iteration on a model of this size takes long. On setA-04 the start phase found nothing better than the baseline within its 180 seconds, and neither did the full model.

The restricted model behaves better than its size would suggest. Thirteen of the fifteen restricted runs end with status optimal, all but one at the root node: on this problem the vertex returned by the simplex method is very often already integral. Over the 20 instances the MLU decreases by 21.7 % on average, by more than 50 % on setA-05, 10, 14 and 17. Seven results are provably optimal for the original problem.

Table 3: Step 6, period 0 only, time limit 1800 s. Model: full model (exact) or restricted model with K candidate waypoints. * provably optimal for the original problem. ‡ HiGHS exceeded the time limit.

Inst	$\lvert V\rvert$	$\lvert A\rvert$	$\lvert D\rvert$	Model	Status	Best obj.	Gap (%)	CPU (s)	Nodes	Baseline	MCF
01	20	80	40	exact	Opt.	0.9294*	–	0.8	1	1.0000	0.5604
02	30	150	45	exact	Opt.	0.5486*	–	3.1	1	1.0000	0.5388
03	50	250	20	exact	Opt.	0.9435*	–	404.6	1	1.0000	0.6337
04	50	250	200	exact	Not opt.	0.6918	33.6	1799.2	0	0.6918	0.3817
05	100	396	100	exact	Not opt.	0.1324	12.8	3050.1‡	0	0.3333	0.1155
06	100	500	500	K=23	Not opt.	1.0000	100.0	1799.8	0	1.0000	0.0986
07	100	500	800	K=11	Opt.	0.9080*	–	3.4	1	0.9080	0.9080
08	150	654	200	K=30	Not opt.	1.0000	99.0	1799.5	0	1.0000	0.3189
09	150	750	200	K=30	Opt.	0.5054	–	27.9	1	0.6111	0.4254
10	150	966	1000	K=16	Opt.	0.4239	–	238.9	1	1.0000	0.0683
11	200	1000	400	K=26	Opt.	0.6446	–	130.9	203	1.0000	0.6131
12	200	898	400	K=26	Opt.	1.0001	–	10.7	1	1.0001	0.5789
13	200	1000	2000	K=11	Opt.	0.6197	–	58.8	1	1.0000	0.0389
14	250	1108	600	K=21	Opt.	0.4397	–	40.2	1	1.0000	0.4239
15	250	1250	600	K=21	Opt.	0.8252*	–	10.9	1	1.0000	0.8252
16	250	1452	4800	K=6	Opt.	0.8730	–	19.2	1	0.9672	0.0386
17	300	1270	2000	K=11	Opt.	0.4242*	–	35.8	1	1.0001	0.4242
18	300	1500	2000	K=11	Opt.	0.6334*	–	10.3	1	0.6334	0.6334
19	300	1998	6000	K=6	Opt.	0.7377	–	25.8	1	1.0000	0.0468
20	400	2000	6000	K=6	Opt.	0.9913	–	12.9	1	1.0000	0.6482

Table 4: Step 9, periods 0 and 1 with the budget constraint, time limit 3600 s. Last column: number of changed segments over the budget κ. * provably optimal for the original problem (full model optimal, objective equal to the MCF bound, or equal to the proven step 6 optimum, which is a lower bound since step 6 is a relaxation of step 9).

Inst	$\lvert V\rvert$	$\lvert A\rvert$	$\lvert D\rvert$	Model	Status	Best obj.	Gap (%)	CPU (s)	Nodes	Baseline	MCF	Chg.
01	20	80	40	exact	Opt.	0.9294*	–	8.6	1	1.0000	0.5604	4/51
02	30	150	45	exact	Not opt.	0.9031	30.9	3594.9	0	1.0981	0.6243	18/63
03	50	250	20	exact	Not opt.	0.9435*	32.8	3597.3	0	1.0000	0.6337	22/53
04	50	250	200	K=20	Opt.	0.7421	–	50.0	1	1.0000	0.4832	0/44
05	100	396	100	exact	Not opt.	0.2050*	100.0	3632.4‡	0	1.0000	0.2050	0/1
06	100	500	500	K=23	Not opt.	1.0000	100.0	3598.6	0	1.0000	0.0986	0/13
07	100	500	800	K=11	Opt.	0.9080*	–	126.8	1	1.0000	0.9080	14/90
08	150	654	200	K=30	Not opt.	1.0000	98.5	3596.5	0	1.0000	0.3189	0/13
09	150	750	200	K=30	Opt.	0.8497*	–	61.6	1	1.0000	0.8497	0/18
10	150	966	1000	K=16	Opt.	0.4239	–	628.6	1	1.0000	0.0683	0/1
11	200	1000	400	K=26	Opt.	0.7858*	–	89.0	1	1.0000	0.7858	5/89
12	200	898	400	K=26	Opt.	1.0001	–	70.4	1	1.0001	0.8799	0/13
13	200	1000	2000	K=11	Opt.	0.6496	–	482.5	1	1.0000	0.0408	9/12
14	250	1108	600	K=21	Opt.	0.5331	–	315.3	1	1.0000	0.5074	0/13
15	250	1250	600	K=21	Opt.	0.8987*	–	71.1	1	1.0000	0.8987	0/54
16	250	1452	4800	K=6	Opt.	0.8811	–	196.0	1	1.0000	0.0441	4/13
17	300	1270	2000	K=11	Opt.	0.4242*	–	328.9	1	1.0001	0.4242	0/1
18	300	1500	2000	K=11	Opt.	1.0000*	–	46.6	1	1.0000	1.0000	3/89
19	300	1998	6000	K=6	Opt.	0.7377	–	249.6	1	1.0000	0.0468	0/13
20	400	2000	6000	K=6	Opt.	0.9913	–	62.4	1	1.0000	0.6482	28/90

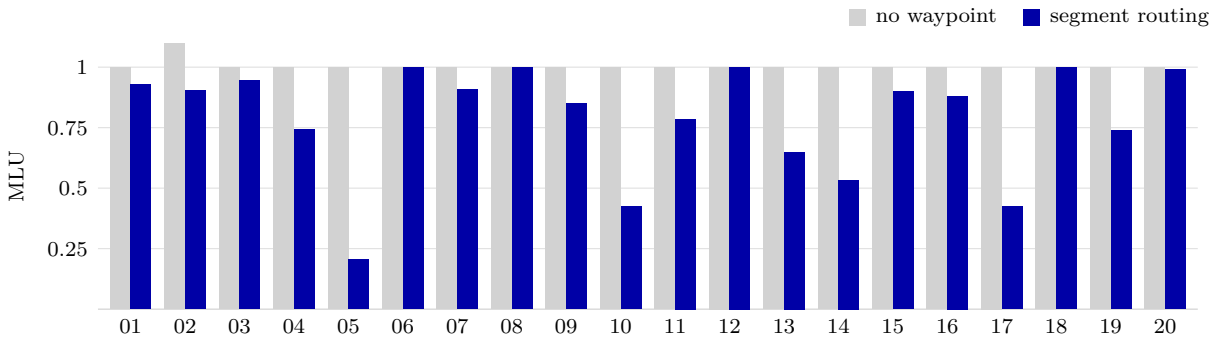


Figure 1: Step 9: maximum link utilization over both periods without waypoints and with the best segment routing found, per instance.

6.3 Two periods with the budget (step 9)

Table 4 and Figure 1 show the two-period results. Removing period 1 and the budget from the step 9 model leaves exactly the step 6 model, so a proven step 6 optimum is a lower bound for step 9. This certifies setA-03: HiGHS reports a 33 % gap, but the solution reaches the step 6 optimum 0.9435 and is therefore optimal. The same argument does not apply to restricted models, whose step 6 value is only an upper bound. Altogether nine step 9 solutions are provably optimal. The full model of setA-04 needed more than 9 GB after half an hour and was stopped by our memory guard; it was solved again with the restricted model ($K = 20$).

The step 9 objective is never below the step 6 one, as expected, and the maximum is attained in period 1 on most instances because of the missing link and the different traffic matrix. The budget is rarely tight. In 11 instances the routing does not change at all between the two periods; only setA-13 comes close to its budget (9 changes out of 12). On setA-05, with $\kappa = 1$, the solution uses no change at all and still lowers the MLU from 1 to 0.205, which matches the MCF bound.

6.4 Why some instances do not improve

Ten of the 40 runs end at the baseline value. They fall into three groups, and telling them apart was one of the most useful parts of the analysis.

The baseline is already optimal. On setA-07 (step 6) and setA-18 (both steps) the baseline equals the MCF bound, so no routing at all can do better. In setA-18 the most loaded arc leaves node 138, which has a single outgoing link; every demand that starts there has to use it, whatever the waypoints.

The candidate set misses the useful waypoint. On setA-12 the restricted model is solved to optimality with MLU 1.0001 while the MCF bound is 0.58. The saturated arc is one of the two arcs entering node 175. The other one starts at node 161 and has a metric about 27 times larger than the surrounding arcs, so node 161 has a large detour and ranks 161st out of 198 candidates for the demands involved. Routing just two of these demands through node 161 by hand gives an MLU of 0.9898. The restricted model is therefore optimal in its own space but misses a better solution; a candidate rule that looks at link loads and not only at detours would fix this case.

The solver does not finish the root LP. This concerns setA-04 in step 6 (see above) and setA-06 and setA-08 in both steps ($K = 23$ and $K = 30$). On the last two the dual simplex was still at the root node when the time limit expired, with no useful bound, so the starting solution was returned. We reran these four cases with the interior point method at the root (Table 5). It always produced a bound, and on setA-08 step 6 it found an optimal solution with MLU 0.3189 in under three minutes, a 68 % reduction. The interior point method is not better everywhere, though: on setA-20 step 9 it reached the bound 0.9913 quickly but found no integer solution below 1 in 20 minutes, while the simplex method solved the same model in 62 seconds. A simplex vertex is often integral

Table 5: Supplementary runs on the two instances where the dual simplex did not finish the root LP: same restricted model, root LP solved by dual simplex (main tables) or by the interior point method.

			Dual simplex			Interior point		
Inst	Step	Model	Best obj.	Bound	CPU (s)	Best obj.	Bound	CPU (s)
06	6	K=23	1.0000	0.0000	1800	1.0000	0.1303	1799
06	9	K=23	1.0000	0.0000	3599	1.0000	0.2535	3600
08	6	K=30	1.0000	0.0098	1799	0.3189	0.3189	172
08	9	K=30	1.0000	0.0147	3596	1.0000	0.3189	3770

on this problem, an interior point is not. Running the simplex first and switching to the interior point method when the root LP stalls would combine the two.

7 Scalability and practical use

The number of binary variables grows as $|D|\, n^2\, |T|$, and memory, not time, is the first limit. With 15 GiB the full model is usable up to about 100 nodes and 100 to 200 demands, which is the size of a regional or metropolitan network, or of a reduced backbone used for planning studies. In that range it gives proven optimal routings, or at least a good solution with a bound, within the time limits we used. The subject allows routing plans to be computed offline, days or weeks ahead, so a run of half an hour to an hour is acceptable in practice.

For national backbones the full model is out of reach, and the restricted model is a reasonable compromise: it produced good solutions for all instances up to 400 nodes and 6000 demands, in a few minutes and with less than 8 GB. Its weak point is the choice of candidates, as setA-12 shows. Two directions seem natural. The first is a column generation scheme in which segment paths are generated by pricing [5]; this keeps the model small without fixing the candidates in advance. The second, cheaper one is to grow the candidate set iteratively around the most loaded arcs. Operationally, our results also suggest that few waypoints are needed: in the solutions most demands use zero, one or two waypoints, and between the two periods the number of changes is far below the budget.

8 Conclusion

We modelled the T-ASR problem as a MILP whose binary variables select segments for each demand and period, with a flow formulation on the complete graph of segments, a segment count limit, load constraints built from shortest-path split ratios, and a linearized change budget. Some reformulations that do not change the model had a large effect on solvability: aggregating the load rows by segment, and splitting the budget row by demand. On the 20 instances of set A, segment routing reduced the maximum link utilization by about 21 % on average and by up to 80 %. Sixteen of the 40 runs are proven optimal; the others are feasible routings with either a solver gap or, for the restricted model, a comparison with the multi-commodity flow bound.

The main limitations are the size of the full model, the heuristic choice of waypoint candidates for large instances, the use of the maximum load instead of the full lexicographic objective, and the single run per instance. The analysis of the failed cases points to concrete improvements: a load-aware candidate rule, a fallback to the interior point method when the root LP stalls, and warm starting the two-period model from the one-period solution.

References

[1] A. Benhamiche, Y. Carlinet, M. Chopin, E. Gourdin, N. Perrot. Challenge EURO/ROADEF 2026, "Keep the Flow!": problem description. Orange Research, 2025.

[2] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, R. Shakir. Segment Routing Architecture. RFC 8402, IETF, 2018.

[3] B. Fortz, M. Thorup. Internet traffic engineering by optimizing OSPF weights. In *Proc. IEEE INFOCOM*, pp. 519–528, 2000.

[4] R. Bhatia, F. Hao, M. Kodialam, T. V. Lakshman. Optimized network traffic engineering using segment routing. In *Proc. IEEE INFOCOM*, pp. 657–665, 2015.

[5] M. Jadin, F. Aubry, P. Schaus, O. Bonaventure. CG4SR: Near optimal traffic engineering for segment routing with column generation. In *Proc. IEEE INFOCOM*, pp. 1333–1341, 2019.

[6] Q. Huangfu, J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

[7] M. Lubin, O. Dowson, J. Dias Garcia, J. Huchette, B. Legat, J. P. Vielma. JuMP 1.0: recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*, 15:581–589, 2023.